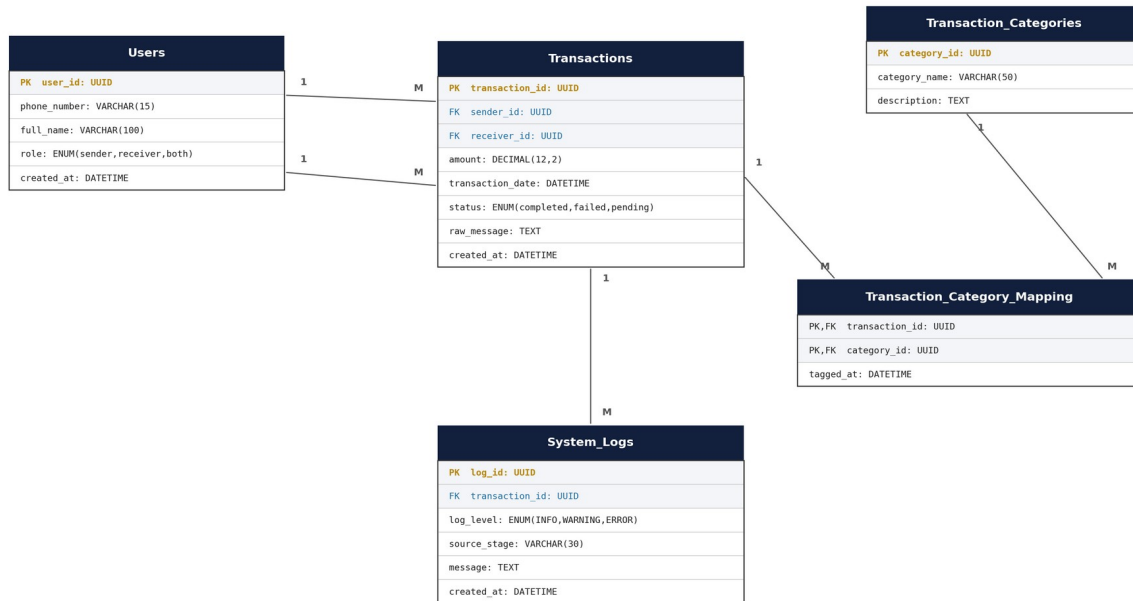


MoMo SMS Analytics

Database Design Document

1. Entity Relationship Diagram



Entities

- **Users** — sender/receiver information (`user_id` PK)
- **Transactions** — main transaction records, linking two Users via `sender_id` and `receiver_id` (both FK)
- **Transaction_Categories** — category lookup (payment types, transfer types, etc.)
- **Transaction_Category_Mapping** — junction table resolving the many-to-many relationship between **Transactions** and **Transaction_Categories**
- **System_Logs** — tracks data processing events per transaction

Relationships

- **Users** → **Transactions** (as sender): one-to-many
- **Users** → **Transactions** (as receiver): one-to-many
- **Transactions** ↔ **Transaction_Categories**: many-to-many, resolved via **Transaction_Category_Mapping**
- **Transactions** → **System_Logs**: one-to-many

2. Design Rationale and Justification

The ERD is built around five entities that map directly onto how a MoMo SMS transaction actually flows through the system. Transactions sits at the center, since every other entity exists to describe, categorize, or track it. Rather than storing sender and receiver as plain text fields, both are modeled as foreign keys pointing to a single Users entity, since a phone number can appear as a sender in one transaction and a receiver in another. This avoids duplicating user details across every transaction and keeps sender/receiver data consistent if it needs to be corrected later.

Transaction_Categories is kept separate from Transactions rather than using a fixed enum, because category types (transfers, payments, airtime, etc.) may need to be added or edited without changing the database schema. A transaction can reasonably need more than one category label at once (for example, a bill payment that is also tagged as a subscription), which is a genuine many-to-many relationship. Rather than forcing a single category_id onto each transaction, we resolved this with a junction table, Transaction_Category_Mapping, which holds paired foreign keys and its own composite key. This keeps the schema normalized instead of repeating transaction data for every category it belongs to.

Finally, System_Logs is linked to Transactions with a one-to-many relationship, since a single transaction can generate multiple log entries as it moves through parsing, cleaning, and loading. Storing logs in their own table rather than appending them to the transaction record keeps the core transaction data clean and makes it possible to trace errors back to the exact processing stage that produced them.

3. Data Dictionary

Every table, column, data type, constraint, and its purpose.

Users

Column	Type	Constraints	Description
user_id	VARCHAR(36)	PRIMARY KEY	Unique identifier for each mobile money user (UUID)
phone_number	VARCHAR(15)	UNIQUE, NOT NULL	User's phone number; unique to prevent duplicate accounts
full_name	VARCHAR(100)	NOT NULL	User's full name
role	VARCHAR(20)	NOT NULL, DEFAULT 'both'	Whether the user acts as sender, receiver, or both
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the user record was created

Transactions

Column	Type	Constraints	Description
transaction_id	VARCHAR(36)	PRIMARY KEY	Unique identifier for each transaction (UUID)
sender_id	VARCHAR(36)	FK → Users.user_id, NOT NULL	The user who sent the money
receiver_id	VARCHAR(36)	FK → Users.user_id, NOT NULL	The user who received the money
amount	DECIMAL(12,2)	NOT NULL, CHECK (amount > 0)	Transaction amount in RWF; must be positive
transaction_date	DATETIME	NOT NULL	When the transaction occurred, per the SMS
status	VARCHAR(20)	NOT NULL, DEFAULT 'pending'	completed / failed / pending
raw_message	TEXT	—	Original SMS body text, kept for traceability
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the record was inserted into the database

Transaction_Categories

Column	Type	Constraints	Description
category_id	VARCHAR(36)	PRIMARY KEY	Unique identifier for each category
category_name	VARCHAR(50)	UNIQUE, NOT NULL	e.g. P2P Transfer, Airtime Purchase, Merchant Payment
description	TEXT	—	Human-readable description of the category

Transaction_Category_Mapping (junction table)

Column	Type	Constraints	Description
transaction_id	VARCHAR(36)	PK, FK → Transactions	Half of the composite key; the tagged transaction
category_id	VARCHAR(36)	PK, FK → Transaction_Categories	Half of the composite key; the applied category
tagged_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When this category tag was applied to the transaction

System_Logs

Column	Type	Constraints	Description
log_id	VARCHAR(36)	PRIMARY KEY	Unique identifier for each log entry
transaction_id	VARCHAR(36)	FK → Transactions, NULLABLE	The related transaction; nullable so history survives if the transaction is deleted
log_level	VARCHAR(20)	NOT NULL, DEFAULT 'INFO'	INFO / WARNING / ERROR
source_stage	VARCHAR(30)	NOT NULL	Pipeline stage that produced this log (e.g. XML_PARSER, DB_INGESTION)
message	TEXT	NOT NULL	Log message content
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the log entry was created

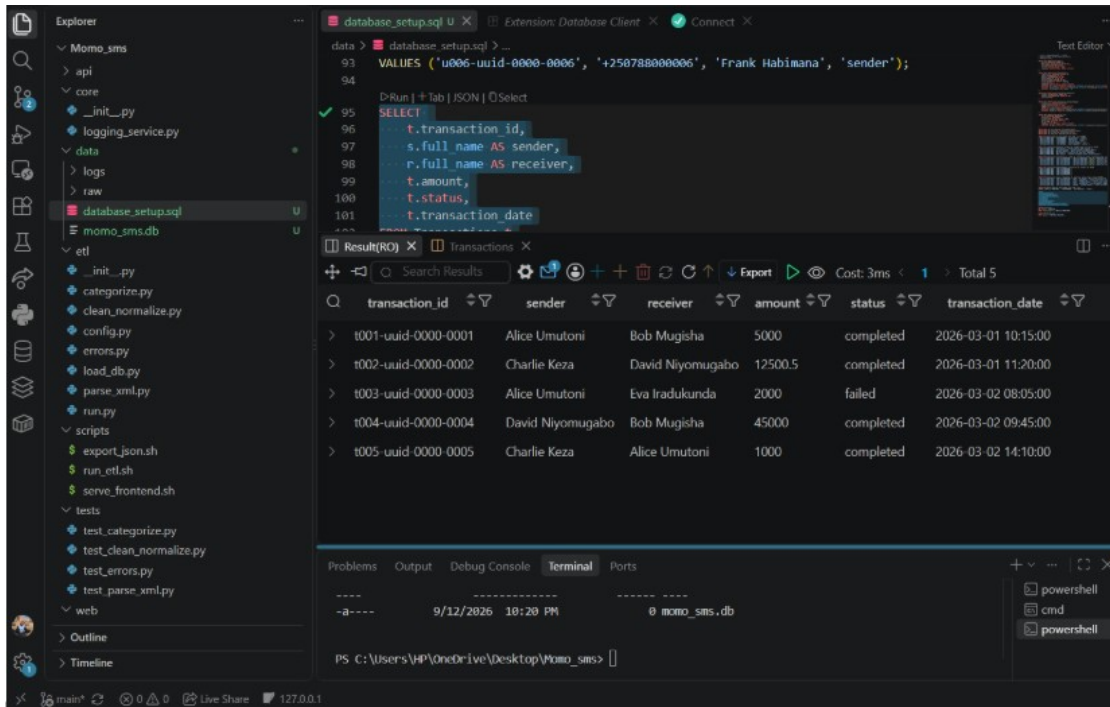
Indexes

- idx_users_phone on Users(phone_number) — speeds up user lookups by phone number
- idx_tx_date on Transactions(transaction_date) — speeds up date-range queries for analytics
- idx_tx_status on Transactions(status) — speeds up filtering by completed/failed/pending
- idx_logs_level on System_Logs(log_level) — speeds up filtering logs by severity

4. Sample Queries Demonstrating Database Functionality

Executed directly against the live database using VS Code's Database Client extension. Real query results are shown below.

Query: transactions joined with sender and receiver names



The screenshot shows the VS Code interface with the Database Client extension. The left sidebar displays the Explorer view with a project structure including folders like 'Momo_sms', 'api', 'core', 'data', 'logs', 'raw', 'etl', and 'tests'. The main editor shows a SQL query in the 'database_setup.sql' file. The query is a SELECT statement joining transactions with sender and receiver information. The results are displayed in a table below the query editor.

```
VALUES ('u006-uuid-0000-0006', '+250788000006', 'Frank Habimana', 'sender');
```

```
SELECT
  t.transaction_id,
  s.full_name AS sender,
  r.full_name AS receiver,
  t.amount,
  t.status,
  t.transaction_date
```

transaction_id	sender	receiver	amount	status	transaction_date
t001-uuid-0000-0001	Alice Umutohi	Bob Mugisha	5000	completed	2026-03-01 10:15:00
t002-uuid-0000-0002	Charlie Keza	David Niyomugabo	12500.5	completed	2026-03-01 11:20:00
t003-uuid-0000-0003	Alice Umutohi	Eva Iradukunda	2000	failed	2026-03-02 08:05:00
t004-uuid-0000-0004	David Niyomugabo	Bob Mugisha	45000	completed	2026-03-02 09:45:00
t005-uuid-0000-0005	Charlie Keza	Alice Umutohi	1000	completed	2026-03-02 14:10:00

Seed data being inserted (categories and transactions)

Explorer

- Momo_sms
 - api
 - core
 - __init__.py
 - logging_service.py
 - data
 - logs
 - raw
 - database_setup.sql
 - momo_sms.db
 - etl
 - __init__.py
 - categorize.py
 - clean_normalize.py
 - config.py
 - errors.py
 - load_db.py
 - parse_xml.py
 - run.py
 - scripts
 - export_json.sh
 - run_etl.sh
 - serve_frontend.sh
 - tests
 - test_categorize.py
 - test_clean_normalize.py
 - test_errors.py
 - test_parse_xml.py
 - web
- Outline

database_setup.sql U Extension: Database Client X Connect X

```
data > database_setup.sql > ...
66 ('c002-uuid-0000-0002', 'Airtime Purchase', 'Airtime recharge via Mobile Money'),
67 ('c003-uuid-0000-0003', 'Merchant Payment', 'Retail shop or service payment'),
68 ('c004-uuid-0000-0004', 'Utility Bill', 'Electricity or water bill settlement'),
69 ('c005-uuid-0000-0005', 'Bank Deposit', 'Transfer from mobile wallet to bank account')
70
D>Run | Select
71 INSERT INTO Transactions (transaction_id, sender_id, receiver_id, amount, transacti
72 ('t001-uuid-0000-0001', 'u001-uuid-0000-0001', 'u002-uuid-0000-0002', 5000.00, '20
73 ('t002-uuid-0000-0002', 'u003-uuid-0000-0003', 'u004-uuid-0000-0004', 12500.50, '20
74 ('t003-uuid-0000-0003', 'u001-uuid-0000-0001', 'u005-uuid-0000-0005', 2000.00, '20
```

Result(RO) X Transactions X

Search Results

Cost: 64ms

DELETE FROM System_Logs WHERE log_id = '1005-uuid-0000-0005'

Problems Output Debug Console Terminal Ports

Directory: C:\Users\HP\OneDrive\Desktop\Momo_sms\data

Mode	LastWriteTime	Length	Name
-a----	9/12/2026 10:20 PM	0	momo_sms.db

PS C:\Users\HP\OneDrive\Desktop\Momo_sms>

5. Unique Rules Enhancing Security and Accuracy

Five constraints built into the schema, each demonstrated below by attempting to break it against the live database.

- UNIQUE constraint on Users.phone_number — prevents duplicate accounts for the same phone number
- CHECK constraint chk_tx_amount — rejects transactions with a zero or negative amount
- FOREIGN KEY ... ON DELETE RESTRICT (Transactions → Users) — blocks deleting a user who still has transaction history, preventing orphaned records
- FOREIGN KEY ... ON DELETE CASCADE (Transaction_Category_Mapping → Transactions) — automatically cleans up category tags when a transaction is removed
- FOREIGN KEY ... ON DELETE SET NULL (System_Logs → Transactions) — preserves log history even after the related transaction is deleted

```
=== Rule 1: UNIQUE constraint blocks duplicate phone numbers ===

sqlite> INSERT INTO Users (user_id, phone_number, full_name, role)
VALUES ('u999-duplicate', '+250788000001', 'Fake Alice', 'sender');
ERROR: UNIQUE constraint failed: Users.phone_number

=== Rule 2: CHECK constraint blocks non-positive amounts ===

sqlite> INSERT INTO Transactions (transaction_id, sender_id, receiver_id, amount, transaction_date,
status)
VALUES ('t999-bad-amount', 'u001-uuid-0000-0001', 'u002-uuid-0000-0002', -500, '2026-03-03
10:00:00', 'pending');
ERROR: CHECK constraint failed: chk_tx_amount

=== Rule 3: Foreign key RESTRICT blocks deleting a user who has transactions ===

sqlite> DELETE FROM Users WHERE user_id = 'u001-uuid-0000-0001';
ERROR: FOREIGN KEY constraint failed

=== Rule 4: Foreign key CASCADE removes category tags when a transaction is deleted ===

sqlite> SELECT COUNT(*) AS mapping_rows_before FROM Transaction_Category_Mapping WHERE
transaction_id = 't003-uuid-0000-0003';
mapping_rows_before
-----
1
(1 rows)

sqlite> DELETE FROM Transactions WHERE transaction_id = 't003-uuid-0000-0003';
Query OK, 1 row(s) affected

sqlite> SELECT COUNT(*) AS mapping_rows_after FROM Transaction_Category_Mapping WHERE transaction_id
= 't003-uuid-0000-0003';
mapping_rows_after
-----
0
(1 rows)

=== Rule 5: Foreign key SET NULL preserves log history after the transaction is gone ===

sqlite> SELECT log_id, transaction_id, message FROM System_Logs WHERE log_id =
't003-uuid-0000-0003';
log_id      transaction_id  message
-----
t003-uuid-0000-0003  None          Insufficient balance on sender wallet
(1 rows)
```